RACIONALIDADE LIMITADA

Luís Morgado
ISEL-DEI

RACIONALIDADE

Racionalidade ilimitada

- **Considera-se:**
  - **Informação completa**
  - **Recursos ilimitados**
- Tipo de mecanismo: **Optimização**
  - Exemplo: Procura A*
- **Não viável no caso geral devido à complexidade computacional dos problemas**

RACIONALIDADE

Racionalidade limitada

- **Considera-se:**
  - **Informação incompleta**
  - **Recursos limitados**
- Tipo de mecanismo: **Satisfação**
  - Processo de decisão orientado para a satisfação de níveis de referência sub-óptimos
  - O processo de decisão pára quando o nível de referência é satisfeito
    - Parametrização do processo de decisão em função dos recursos computacionais disponíveis (tempo, memória)
    - Qualidade da solução *vs.* recursos utilizados

RACIONALIDADE LIMITADA

- **RESTRIÇÕES**
  - Complexidade computacional
  - Recursos limitados
- **MÉTODOS PARA LIDAR COM A COMPLEXIDADE**
  - Aproximação
  - Abstracção
  - Modularização
  - Focagem



APROXIMAÇÃO

- **MÉTODOS HEURÍSTICOS**
  - Respostas **imprecisas** mas **expeditas**
    - Por **eliminação de restrições** do domínio do problema
    - Por exploração de **regularidades** no ambiente
    - Por **memória** de situações relacionadas
    - Por **frequência de ocorrência**
    - Por **categorização**
    - Por **ancoragem** a um referencial prévio
  - Plausibilidade biológica
    - Heurísticas simples e rápidas
      - *Fast and frugal heuristics* [Gigerenzer, 1999]
    - Respostas de natureza afectiva

HEURÍSTICA

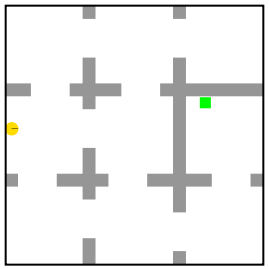
- **Aproximação** para resolução de um problema de forma expedita, **com base em informação sobre o domínio do problema**
- Tem por objectivo **tornar viável a resolução de um problema com restrições de recursos** (tempo, memória), quando isso não é possível com um método óptimo
- **Tem inerente um compromisso entre resposta expedita e**:
  - **Soluções sub-óptimas**
  - **Não completude**
  - **Exactidão**
  - **Precisão**

FUNÇÃO HEURÍSTICA

- Representa uma **estimativa do custo ou utilidade** de uma determinada situação ou acção para a resolução de um problema
- Reflecte **conhecimento acerca do domínio do problema**
- O seu valor é uma **aproximação ao valor real** da medida em causa
- **Exemplo**
  - Avaliação de um estado em relação ao objectivo a atingir
    - Independente do percurso até ao estado
- **Heurística admissível**
  - A estimativa de custo é sempre inferior ou igual ao custo efectivo mínimo
    - **Optimista**

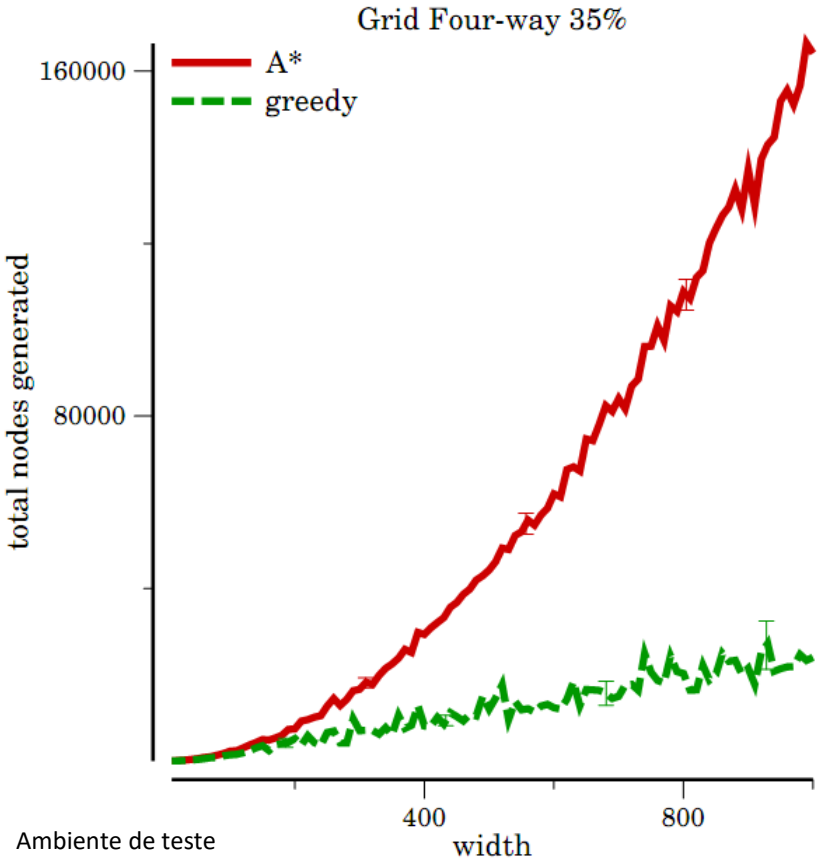
FUNÇÃO HEURÍSTICA

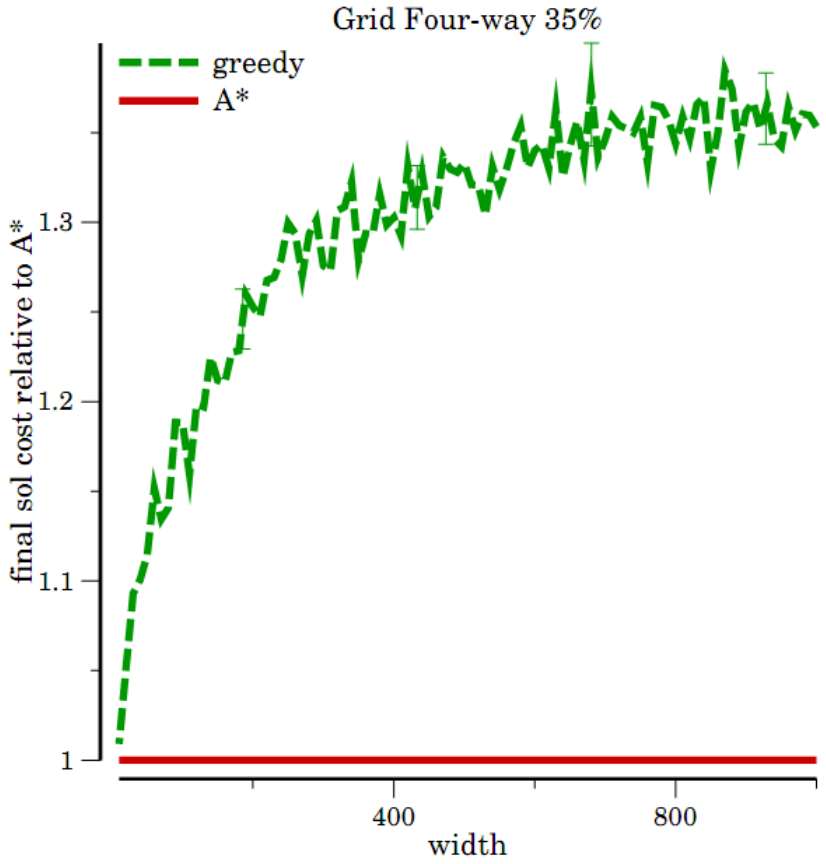
- Como definir uma **heurística admissível**
  - No caso geral, uma heurística admissível é obtida através do **relaxamento de restrições** associadas ao problema
  - Exemplo: Navegação autónoma
    - h_1 - Distância de Manhattan
      - Corresponde a retirar a restrição:
        - » Não movimentação através de obstáculos
    - h_2 - Distância de Euclidiana
      - Corresponde a retirar as restrições:
        - » Não movimentação através de obstáculos
        - » Não movimentação em diagonal



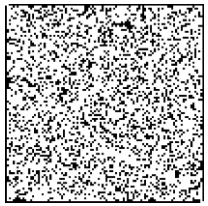
ESFORÇO vs. QUALIDADE DA SOLUÇÃO

Regulação do efeito da heurística na relação entre esforço computacional e qualidade da solução





Ambiente de teste



[Thayer & Ruml, 2007]

PROCURA A* PONDERADA (*WEIGHTED A**)

Algoritmo de procura *ε-Óptimo*

Garante um custo máximo da solução encontrada:

$$f(t) = (1 + \varepsilon).\delta(s, T)$$

ε – parâmetro de controlo

s – estado inicial

$t \in T$ – estado terminal

T – conjunto de estados terminais

$\delta(s, T)$ – custo da solução óptima

Critério de procura para garantir solução *ε*-óptima:

Procura A* com:

$$f(u) = g(u) + (1 + \varepsilon).h(u)$$

Sendo $h(u)$ uma heurística admissível

PROCURA A* PONDERADA (*WEIGHTED A**)

Algoritmo de procura *ε-Óptimo*

Garante um custo máximo de uma solução encontrada:

$f(t) = (1 + \varepsilon).\delta(s, T)$

$f(u) = g(u) + (1 + \varepsilon).h(u)$

ε – parâmetro de controlo

s – estado inicial

$t \in T$ – estado terminal

T – conjunto de estados terminais

δ – custo óptimo

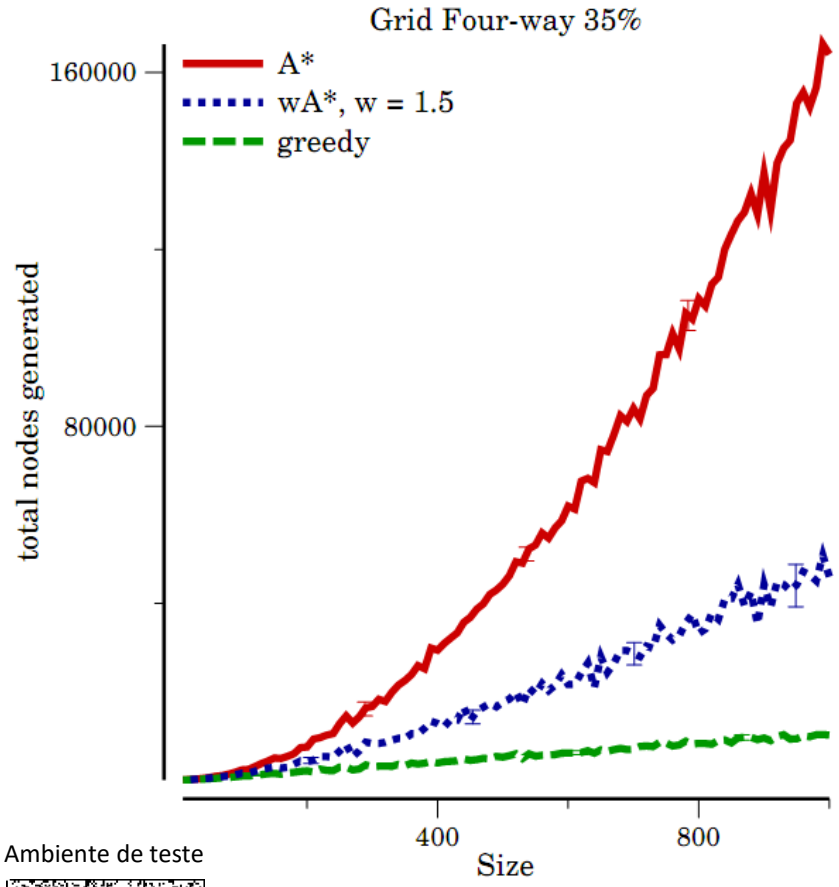
Prova:

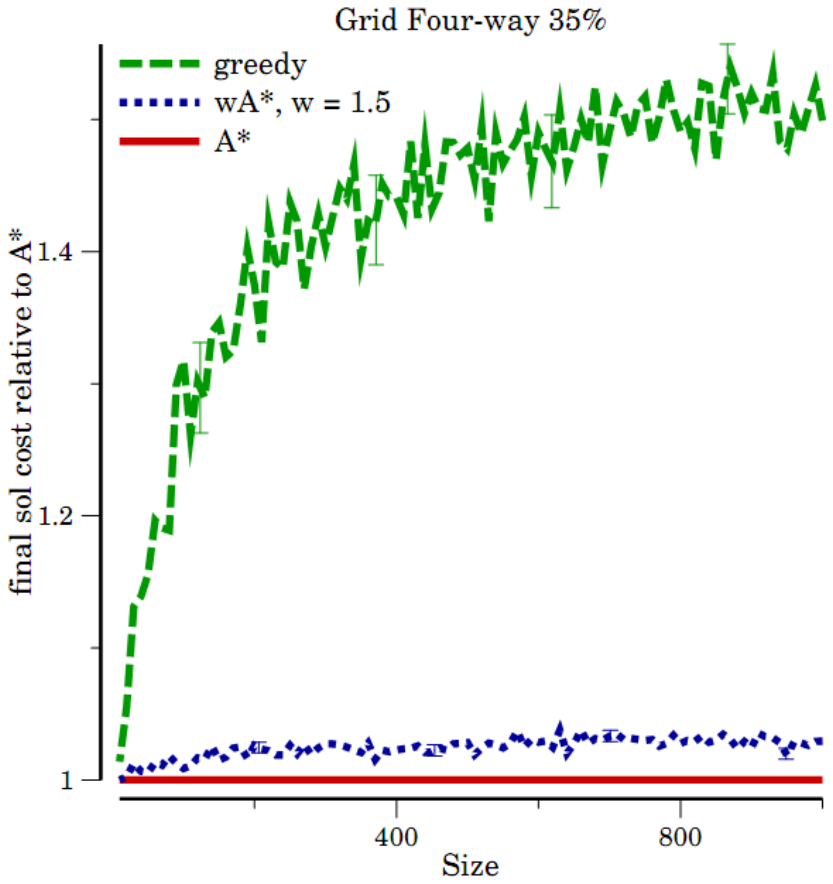
$$
\begin{aligned}
f(u) &\leq \delta(s, u) + (1 + \varepsilon).\delta(u, T) && ; \text{mínimo se } u \text{ é um nó no percurso óptimo} \\
&\leq \delta(s, u) + \delta(u, T) + \varepsilon.\delta(u, T) \\
&\leq \delta(s, T) + \varepsilon.\delta(u, T) && ; \delta(s, u) + \delta(u, T) = \delta(s, T) \\
&\leq \delta(s, T) + \varepsilon.\delta(s, T) && ; \delta(u, T) \leq \delta(s, T) \\
&\leq (1 + \varepsilon).\delta(s, T)
\end{aligned}
$$

Logo, se um nó $t \in T$ for seleccionado, $f(t) = (1 + \varepsilon).\delta(s, T)$

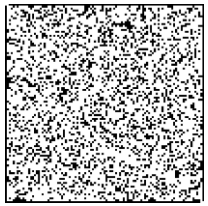
PROCURA A* PONDERADA (*WEIGHTED A**)

Regulação do efeito da heurística na relação entre esforço computacional e qualidade da solução





Ambiente de teste

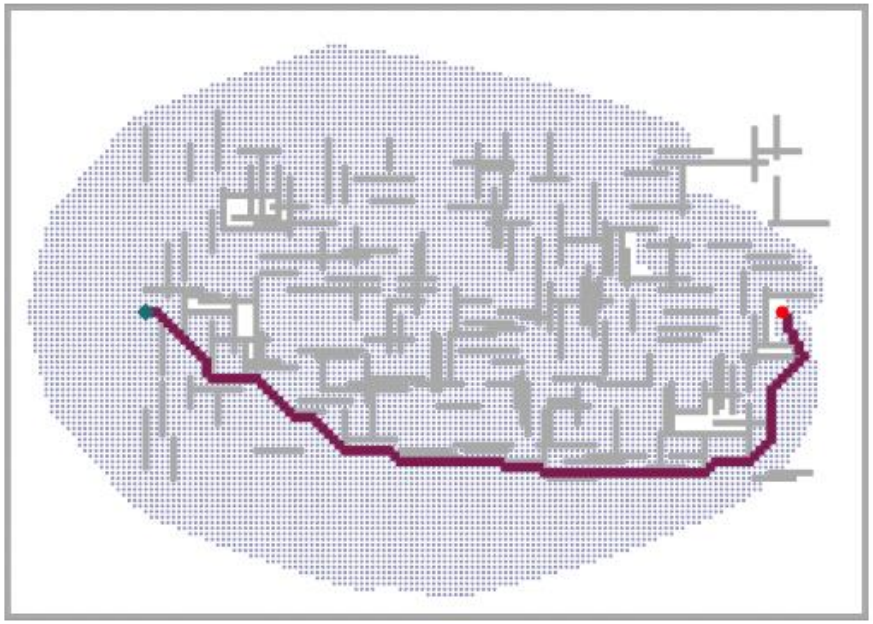


[Thayer & Ruml, 2007]

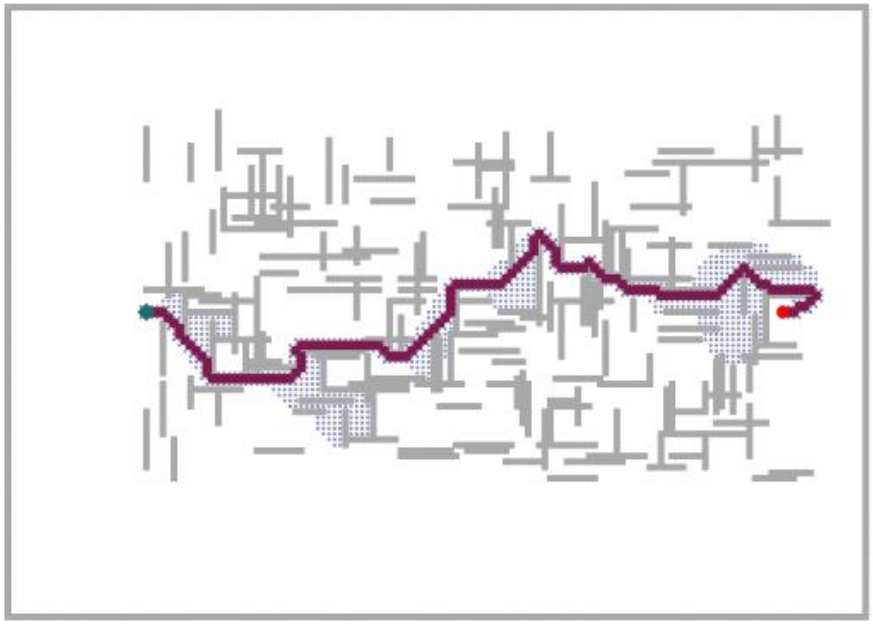
PROCURA A* PONDERADA (*WEIGHTED A**)

Exemplo:

Procura A*



Procura A* ponderada



Procura A* ponderada:

Exploração de 7 vezes menos estados com apenas 5% de aumento do custo da solução

[Russel & Norvig, 2022]

PROCURA A* PONDERADA (*WEIGHTED A**)

IMPLEMENTAÇÃO

Iniciar mecanismo de procura definindo o valor do parâmetro de controlo ε

```
1. function iniciar(ε)
2.    set ε = ε
```

set propriedade = valor (atribuição de valor a propriedade)

PROCURA A* PONDERADA (*WEIGHTED A**)

Função de custo: $f(n) = g(n) + (1 + \varepsilon).h(n)$

```
1. function f(no): float
2.    return no.custo +
3.           (1 + ε) * problema.heuristica(no.estado)
```

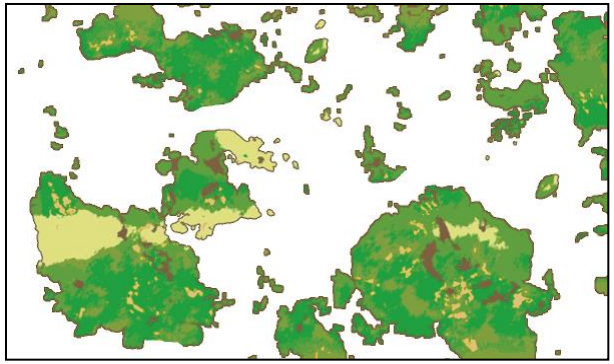
ABSTRACÇÃO

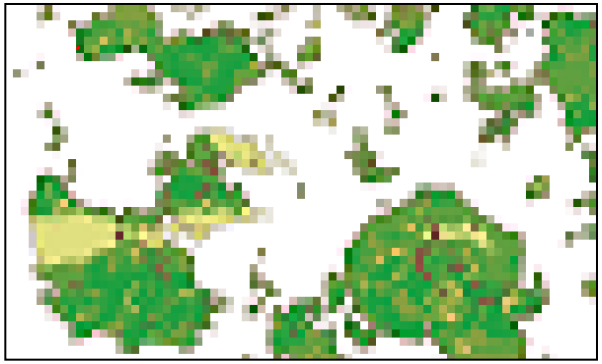
O processo de descrição de conhecimento a *diferentes níveis de detalhe* (quantidade de informação) e *tipos de representação* (estrutura da informação).

[Korf, 1980]

ABSTRACÇÃO

- **Redução de detalhe**
  - Identificação de *características principais*
  - Redução de complexidade da representação
  - Redução de complexidade do processamento

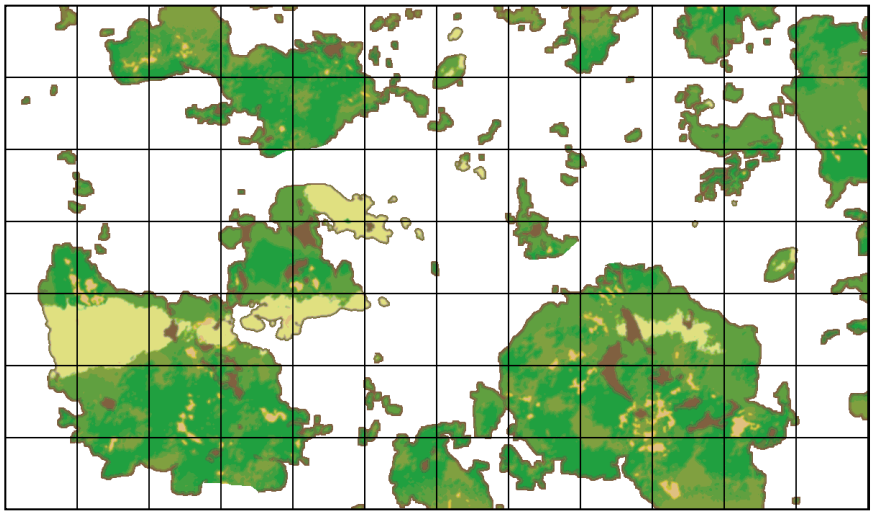


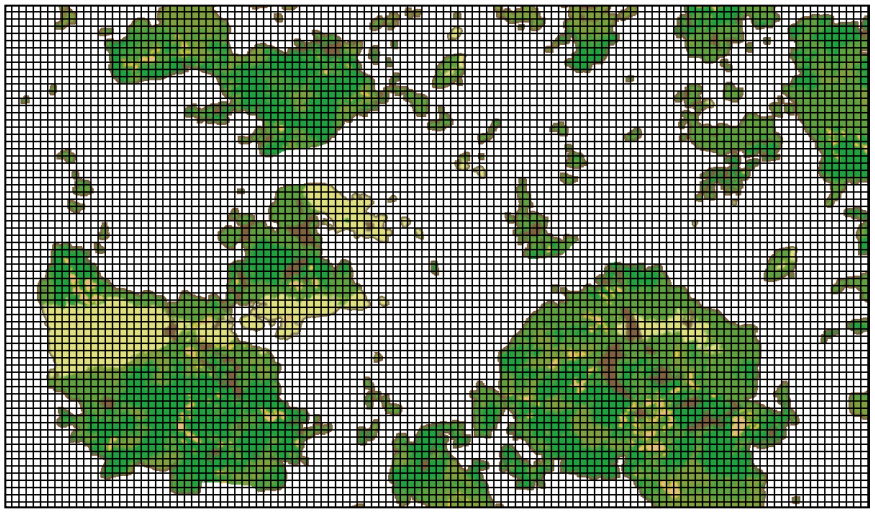


- **Problemas**
  - Qualidade da solução
  - Método não completo
    - Pode não ser possível encontrar solução, apesar de existir solução em níveis de detalhe superior

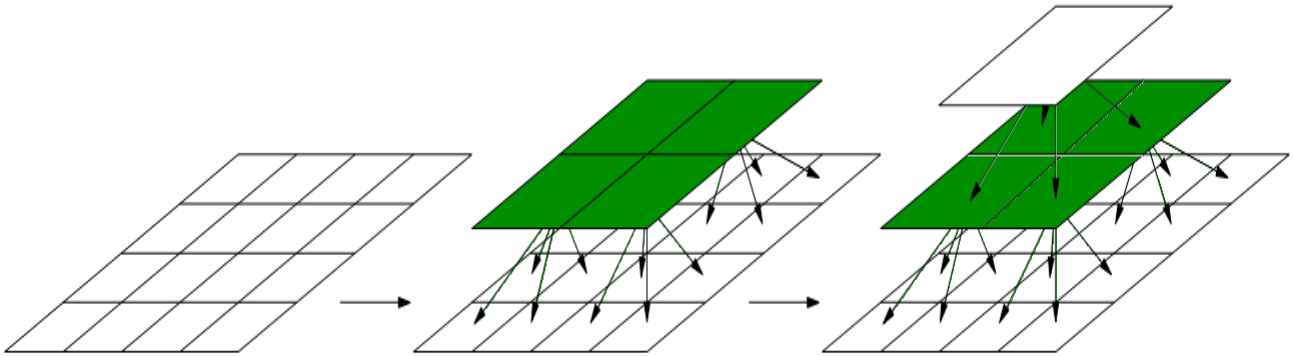
ABSTRACÇÃO LINEAR

ABSTRACÇÃO LINEAR EM GRELHA





ABSTRACÇÃO LINEAR EM GRELHA HIERÁRQUICA



[Sturtevant & Buro, 2005]

ABSTRACÇÃO NÃO LINEAR

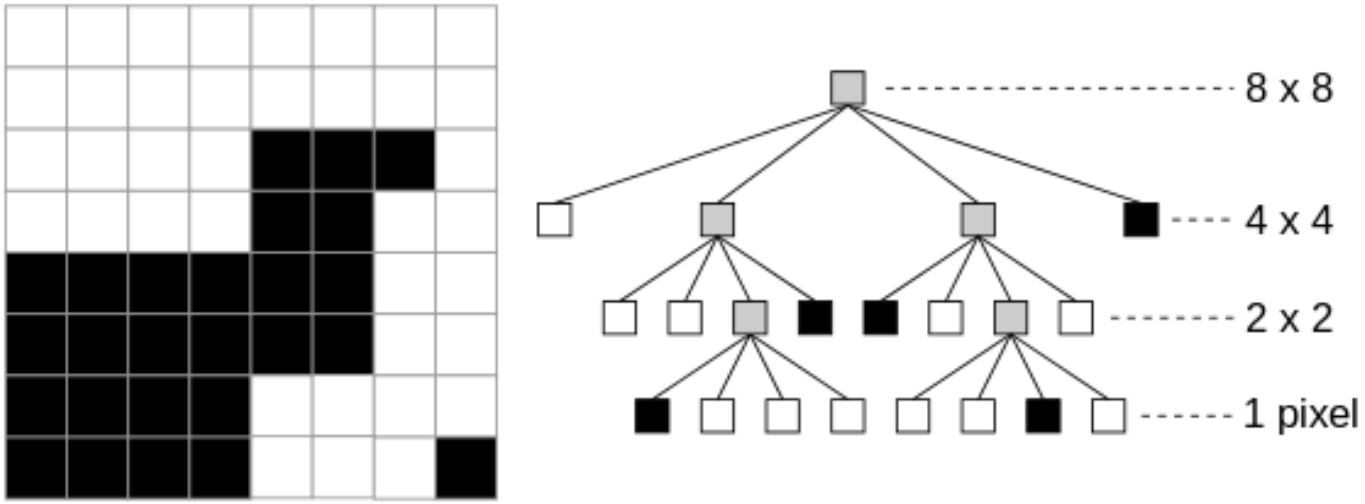
Estruturas *Quadtree* e *KD-Tree*

Estruturas de dados em árvore utilizadas para abstrair informação em espaços bidimensionais (*quadtree*) ou em espaços *n*-dimensionais (*kd-tree*)

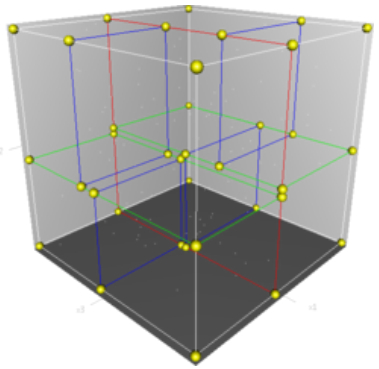
- No caso da estrutura *quadtree* o espaço bidimensional é sucessivamente dividido em regiões menores, simétricas, designadas *quadrantes*
- Cada quadrante pode ser subdividido recursivamente em novos quadrantes, caso exista informação relevante para representar nessa região

ABSTRACÇÃO NÃO LINEAR

Quadtree - Abstracção não-linear em espaços bidimensionais

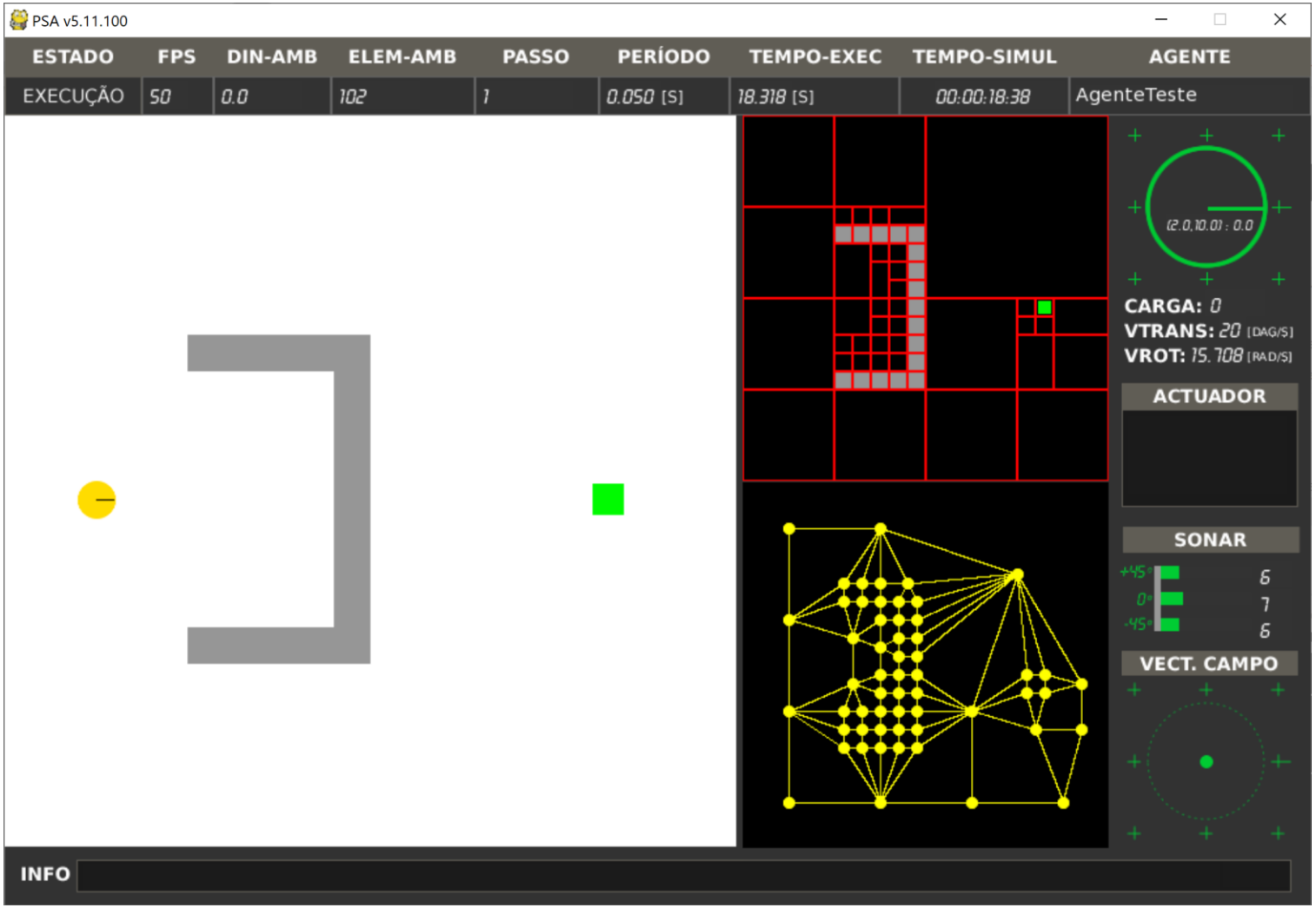


KD-Tree - Abstracção não-linear em espaços *n*-dimensionais



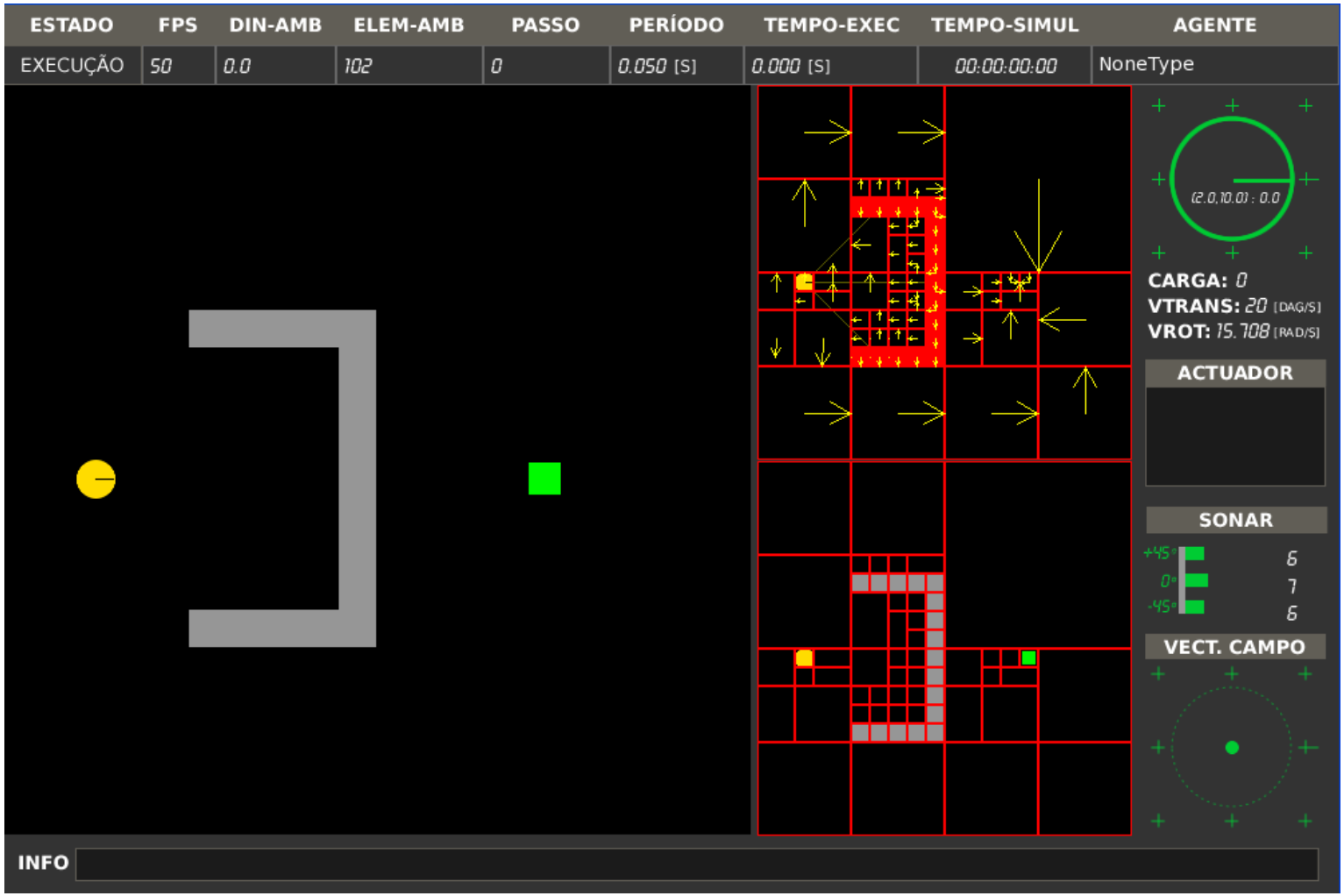
ABSTRACÇÃO COM BASE EM *QUADTREES*

Exemplo: Utilização de uma estrutura *quadtree* para abstracção de um espaço bidimensional



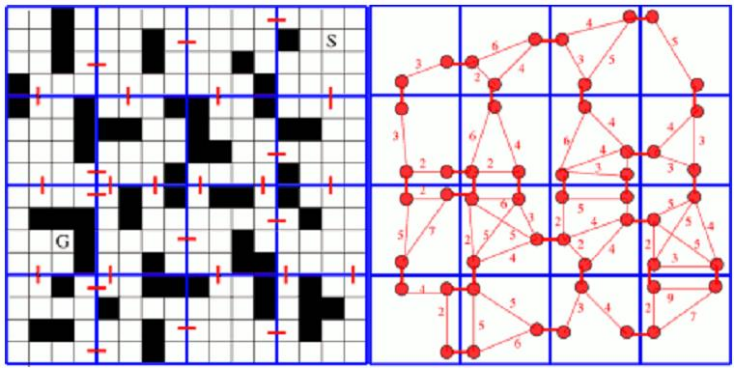
ABSTRACÇÃO COM BASE EM *QUADTREES*

Exemplo: Utilização de uma estrutura *quadtree* num problema de planeamento num espaço bidimensional

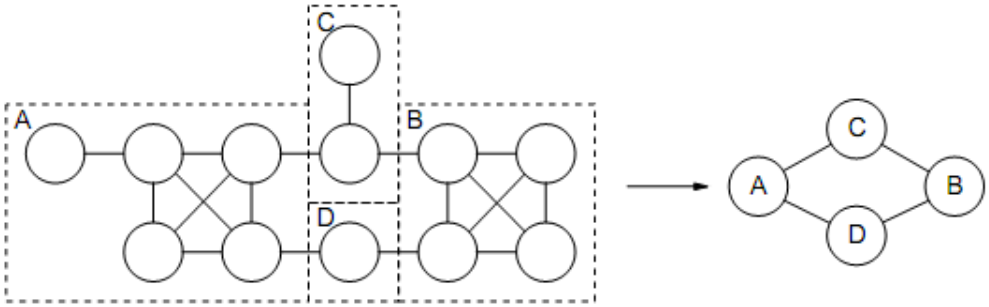


ABSTRACÇÃO COM BASE EM GRAFOS

Abstracção com base em grafos gerados a partir de conhecimento do domínio do problema



Abstracção em grafo baseada em pontos-guia



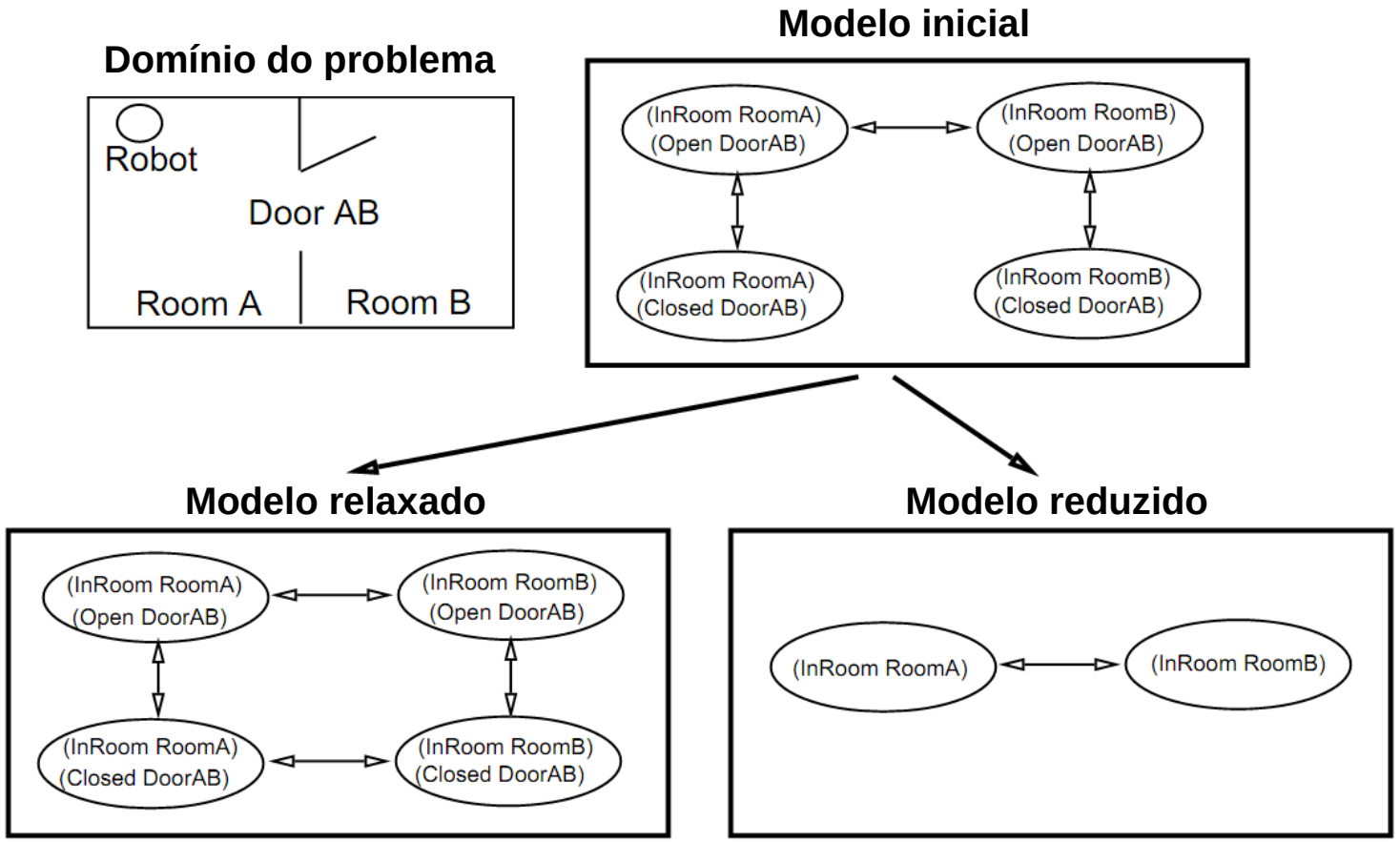
Abstracção hierárquica de grafos

A capacidade de abstracção depende de conhecimento do domínio do problema

ABSTRACÇÃO SIMBÓLICA

Utiliza representações simbólicas de conhecimento do domínio do problema

- Redução de detalhe, com foco na informação relevante para o raciocínio
- Abstracção através do relaxamento de restrições nos modelos mais abstractos

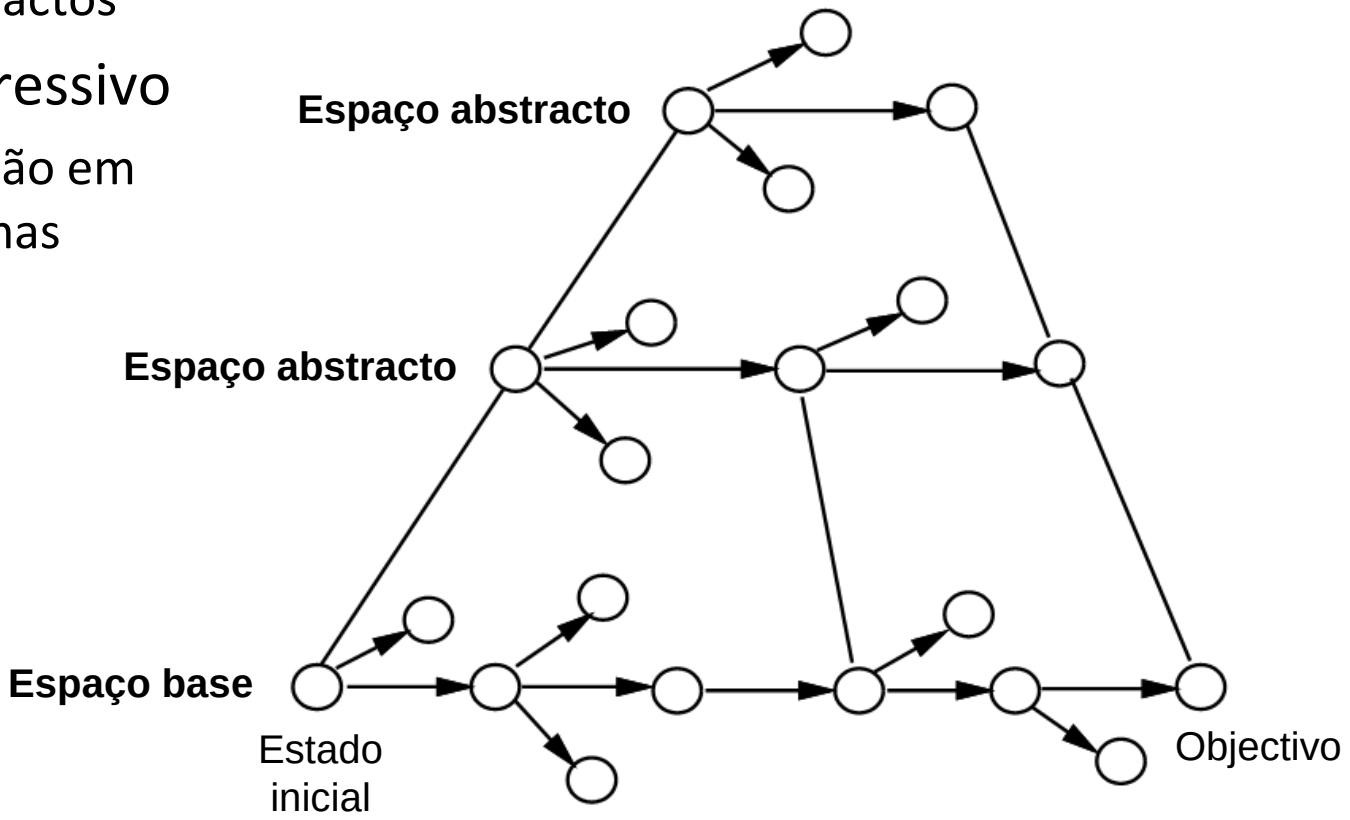


Adaptado de [Knoblock, 1991]

ABSTRACÇÃO SIMBÓLICA

Planeamento hierárquico

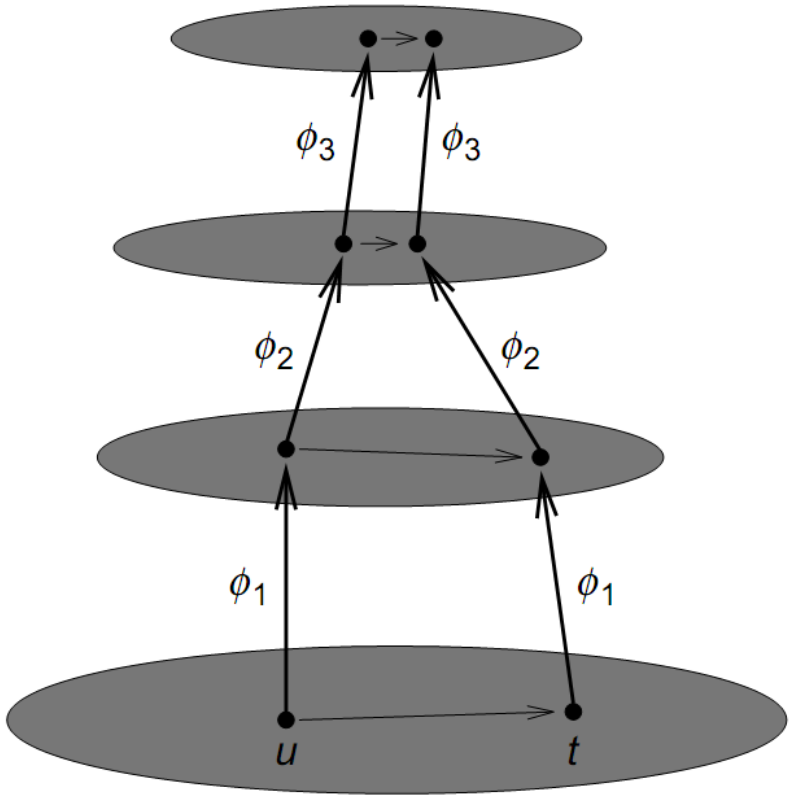
- Estratégias de alto nível
  - Planos abstractos
- Detalhe progressivo
  - Decomposição em sub-problemas



Adaptado de [Knoblock, 1991]

PROCURA A* HIERÁRQUICA

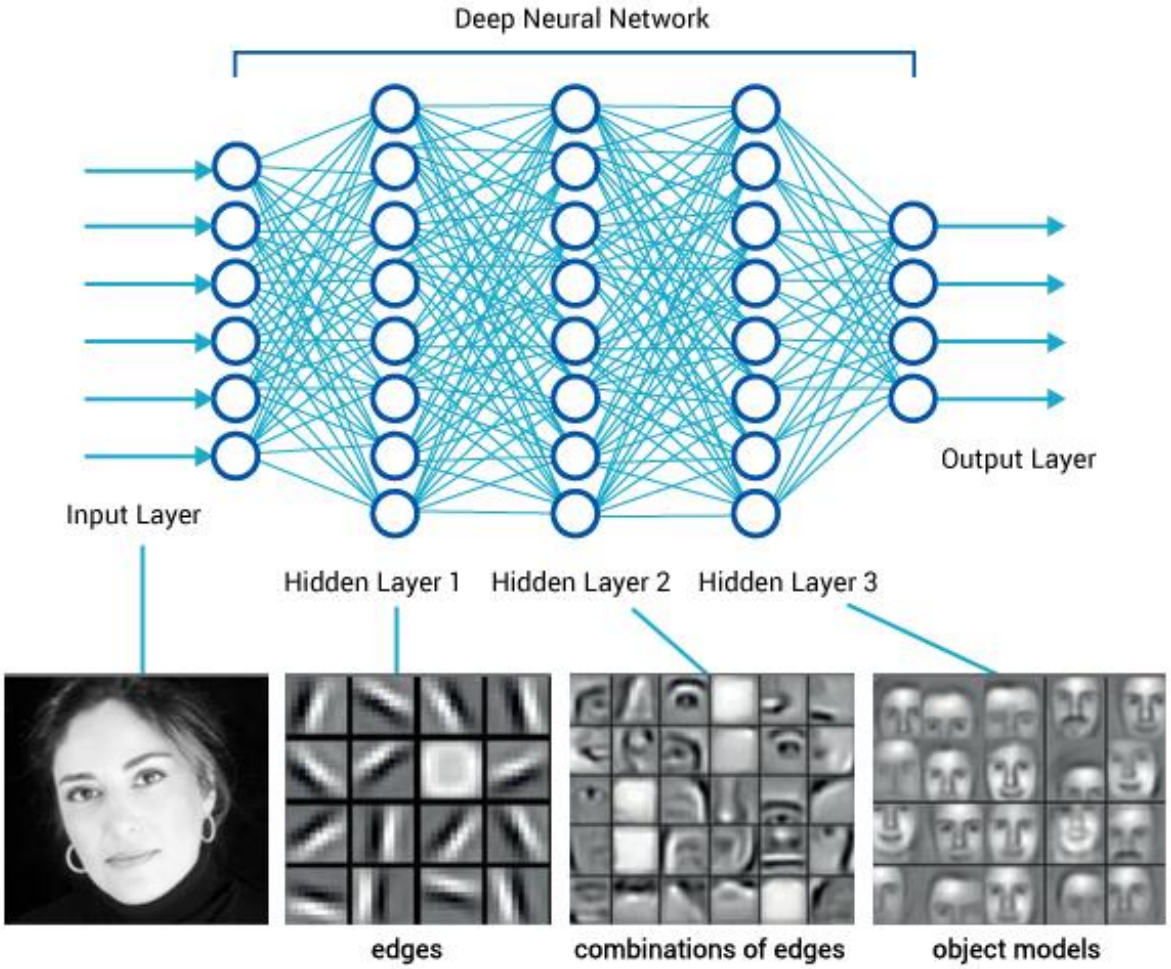
- Níveis de transformação
  - $\phi_1,\ \phi_2,\ \phi_3, \ldots$
- Função heurística de um nível gerada a partir de níveis de abstração superiores
  - Exemplo
    - $\phi_2(\phi_1(u)),\ \phi_2(\phi_1(t))$
- Função heurística não consistente
- Pré-computação da função heurística a níveis abstractos para optimização de desempenho



[Edelkamp & Schrodl, 2012]

ABSTRACÇÃO COM BASE EM REDES NEURONAIS

A capacidade de generalização/abstracção das redes neuronais artificiais pode ser utilizada para abordar o problema da representação em problemas de grande complexidade, em particular com base em redes neuronais profundas



MODULARIZAÇÃO

- Associada à abstracção
- Redução de complexidade por decomposição combinatória
  - divisão em sub-problemas com relação causal
  - Resolução sequencial
  - Soluções parciais
  - Composição da solução global em função das soluções parciais
- Exemplo
  - Navegação através de pontos de referência (*waypoints*)



EXEMPLO: *15-PUZZLE*

$N \approx 10^{12}$

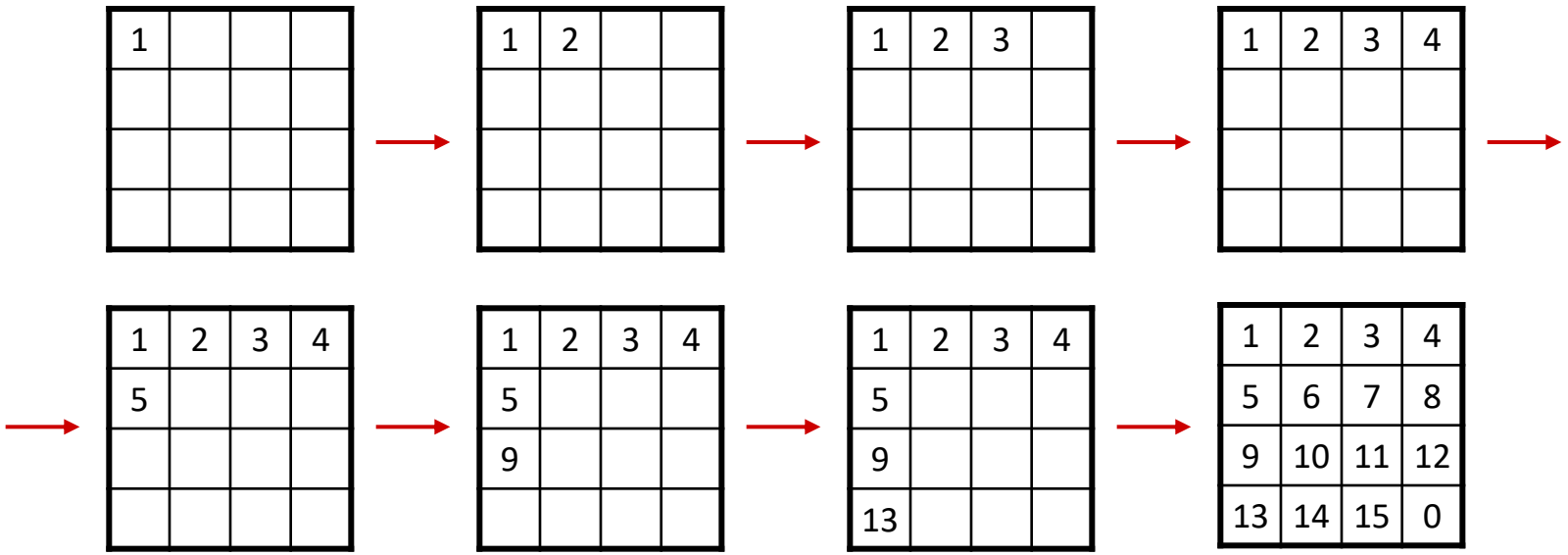
9	2	12	6
5	7	14	13
3	4	1	11
15	10	8	0



1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	0

Plano abstracto guia raciocínio concreto

- Decomposição em sub-problemas
- Resolução independente de cada sub-problema



REFERÊNCIAS

[Korf, 1980]

R. Korf, *Toward a model of representation changes, Artificial Intelligence*, Volume 14, Issue 1, 1980

[Edelkamp & Schrodl, 2012]

S. Edelkamp, S. Schrodl, *Heuristic Search Theory and Applications*, Morgan Kaufmann, 2012

[Pearl, 1984]

J. Pearl, *Heuristics: Intelligent Search Strategies for Computer Problem Solving*, Addison-Wesley, 1984

[Murphy, 2000]

R. Murphy, *An Introduction to AI Robotics*, MIT Press, 2000

[Thayer & Ruml, 2007]

J. Thayer, W. Ruml, *A Survey of Suboptimal Search Algorithms*, University of New Hampshire, 2007

[Sturtevant & Buro, 2005]

N. Sturtevant, M. Buro, *Partial Pathfinding Using Map Abstraction and Refinement*, Proceedings of the 20th national conference on artificial intelligence (AAAI-05), 2005

[Botea *et al.*, 2004]

A. Botea M. Muller, J. Schaeffer, *Near Optimal Hierarchical Path-Finding*, Journal of Game Development, Volume 1, 2004

[Cowlagi , Tsiotras, 2007]

R. Cowlagi , P. Tsiotras, *Beyond Quadtrees: Cell Decompositions for Path Planning using Wavelet Transforms*, Proceedings of the 46th IEEE Conference on Decision and Control, 2007

[Knoblock, 1991]

A. Knoblock, *Automatically Generating Abstractions for Problem Solving*, PhD Thesis, Carnegie Mellon University, 1991

[Russel & Norvig, 2022]

S. Russel, P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2022

[Trajan, 1983]

R. Trajan, *Data Structures and Network Algorithms*, Bell Laboratories, 1983

[Sipser, 2005]

M. Sipser, *Introduction to the Theory of Computation*, Thomson, 2005